Processus internes

Livrable du projet T-ESP-901

Réalisé par Justin MBODJE, Yasser MEKHFI, Jérémy COQUELIN, Mohamed FOFANA, Mattéo COGAN, Romain CROCQ-MARCHAND, Tristan CHARDES

Établissement de formation : EPITECH RENNES

Promotion 2026

Sommaire

Processus internes – Projet Corevia (ESP)..... 3
1. Objectif..... 3
2. Gestion des tâches.....3
3. Process de développement.....3
4. Gestion des connaissances..... 4
5. Intégration d'un nouveau membre.....4
6. Départ d'un membre.....5
7. Process de recrutement..... 5
8. Gestion des blocages.....6
9. Communication..... 6
10. Fonctionnement général.....6

Processus internes – Projet Corevia (ESP)

1. Objectif

Ce document formalise les principaux processus utilisés par l'équipe afin d'assurer la continuité du projet, la qualité du développement et une organisation cohérente.
Il ne remplace pas les documents d'organisation existants, mais précise concrètement comment les actions sont réalisées au quotidien.

2. Gestion des tâches

Les tâches sont définies dans le backlog **Jira** et priorisées lors du **sprint planning** en début de sprint. Le projet suit une méthodologie **Agile Scrum avec des sprints de 2 semaines**.

Avant de commencer, le développeur vérifie que la tâche est suffisamment claire et qu'elle répond aux critères d'acceptation définis. Si ce n'est pas le cas, elle est précisée rapidement avec le chef de projet ou le lead concerné.

Lorsqu'une tâche est prise, elle est assignée à une seule personne afin d'éviter les responsabilités floues. Le statut est mis à jour régulièrement dans Jira (à faire / en cours / en review / terminée).

Une tâche est considérée comme terminée lorsqu'elle respecte la **Definition of Done** du projet (tests, relecture, documentation, déploiement en recette).

3. Process de développement

Chaque développement suit le cycle défini dans le Plan qualité.

Avant toute implémentation, une branche dédiée est créée depuis la branche de stack correspondante, selon la convention `feature/`, `bugfix/` ou `hotfix/`. Le code est produit de manière incrémentale avec des commits compréhensibles.

Avant d'ouvrir une merge request, le développeur s'assure que :

- le code fonctionne sans erreur bloquante en local
- les tests unitaires couvrant la fonctionnalité sont écrits et passent
- le comportement correspond à la tâche et à ses critères d'acceptation
- la documentation associée (code, API) est à jour

La merge request est ensuite ouverte. Elle doit être validée par :

- **Un développeur**
- **Code Rabbit** (revue automatisée)
- la CI/CD doit être entièrement passée (tests + build)

Pour une merge sur `main`, l'approbation du **chef de projet** est également requise.

4. Gestion des connaissances

Afin d'éviter la dépendance à une seule personne, chaque fonctionnalité développée doit rester compréhensible par le reste de l'équipe.

Les choix techniques importants sont documentés de manière simple.
Le code doit être lisible et structuré pour permettre une reprise rapide si nécessaire.

En cas de sujet complexe, une explication rapide peut être faite à l'oral ou ajoutée dans la documentation existante.

La documentation technique de chaque stack est maintenue à jour dans son dépôt (format Markdown), et la documentation de l'API est générée via **Swagger**.

5. Intégration d'un nouveau membre

L'arrivée d'un nouveau membre suit un processus structuré mais pragmatique.

Dans un premier temps, il reçoit les accès aux outils : **GitHub, Jira, Discord, services cloud**, et les éventuels accès spécifiques à sa stack.

Il prend connaissance des documents de référence du projet : la **Synthèse Corevia**, le **Plan qualité**, les **Spécifications fonctionnelles**, et la **Definition of Done**.

Une première tâche simple (bug mineur ou amélioration UI) lui est attribuée pour lui faire découvrir la codebase et les pratiques de l'équipe. Il est accompagné par le lead de sa stack pendant cette phase.

L'accompagnement se fait de manière progressive en fonction de son autonomie, et un point est fait avec le chef de projet à la fin du premier sprint.

6. Départ d'un membre

Lorsqu'un membre quitte le projet, une attention particulière est portée à la transmission des informations.

Ses tâches en cours sont soit finalisées, soit réassignées.
Les éléments critiques sur lesquels il travaillait doivent être compréhensibles par le reste de l'équipe.

Si nécessaire, un court transfert de connaissance est réalisé pour éviter toute perte d'information.

7. Process de recrutement

Le recrutement se fait de manière pragmatique.

Un premier échange permet d'évaluer rapidement la motivation et la disponibilité du candidat.
Une vérification informelle de ses compétences est réalisée via discussion ou expérience passée.

Une phase d'intégration est ensuite mise en place via une tâche concrète.
Cela permet d'évaluer sa capacité à travailler dans le contexte du projet (compréhension, communication, autonomie).

La décision d'intégration repose autant sur l'aspect technique que sur la capacité à s'intégrer dans l'équipe.

8. Gestion des blocages

Lorsqu’un blocage apparaît, il doit être signalé rapidement.

L’équipe privilégie une courte résolution : échange rapide, aide ponctuelle ou ajustement de la tâche.
Si le blocage persiste, une décision est prise collectivement pour débloquer la situation.

Les blocages sont **tracés dans Jira** (statut "bloqué" + commentaire) pour qu'ils soient visibles et permettre au chef de projet de prioriser leur résolution.

9. Communication

La communication est volontairement simple et directe.

Les échanges au quotidien se font sur **Discord** (salons dédiés par stack et par sujet). Les décisions importantes et les spécifications sont consignées sur **Jira** ou dans la documentation pour traçabilité.

Un membre ne doit pas rester bloqué plus d'une demi-journée sans en informer son lead ou le chef de projet.

10. Fonctionnement général

Les processus définis ici ont pour objectif de structurer le travail sans le complexifier.

L’équipe privilégie des méthodes simples, adaptables et efficaces.
L’objectif principal reste la progression du projet, la qualité des livrables et la capacité à s’adapter rapidement aux contraintes.